# Practical 11 : Develop a Simple Recommendation System

## # 1. PROJECT OVERVIEW & APPLICATION DOMAIN

- **Student Name:** Ankit raj sharma
- **Enrollment No.:** 92400103142
- **Practical Title:** Concept demonstrated in Practical 11
- **Task:** Self-Learning Task 3
- **Selected Domain:** Movie Recommendation System
- **Core Technology:** Python 3, Pandas DataFrame
- **Project Purpose:** Process structured tabular movie data using Pandas DataFrame to recommend Top 3 and Top 5 items based on rating, user genre preference, and popularity metrics.

## # 2. TECHNOLOGY STACK & ARCHITECTURE

The system is developed using Python data processing libraries following a modular structure:

- **Language:** Python 3.10+ (Core sorting, filtering, and scoring algorithms).
- **Data Manipulation:** Pandas (`pandas.DataFrame`) for dataset representation, boolean conditional indexing, and multi-attribute sorting.
- **Dataset Storage:** `movies_dataset.csv` containing 15 items and 6 attributes.

## # 3. DATASET DESCRIPTION & ATTRIBUTES

The dataset contains **15 movie records** (satisfying the ≥ 10 requirement) with **6 key attributes** (satisfying the 2–4 attributes requirement):

| ID | Title | Genre | Rating | Votes | Year |
|---|---|---|---|---|---|
| 1 | Inception | Sci-Fi | 8.8 | 2,400,000 | 2010 |
| 2 | The Dark Knight | Action | 9.0 | 2,700,000 | 2008 |
| 3 | Interstellar | Sci-Fi | 8.7 | 1,900,000 | 2014 |
| 4 | Parasite | Drama | 8.5 | 850,000 | 2019 |
| 5 | Spirited Away | Animation | 8.6 | 790,000 | 2001 |
| 6 | Avengers: Endgame | Action | 8.4 | 1,180,000 | 2019 |
| 7 | The Shawshank Redemption | Drama | 9.3 | 2,750,000 | 1994 |
| 8 | Whiplash | Drama | 8.5 | 900,000 | 2014 |
| 9 | Spider-Man: Spider-Verse | Animation | 8.4 | 590,000 | 2018 |
| 10 | The Matrix | Sci-Fi | 8.7 | 1,950,000 | 1999 |
| 11 | Gladiator | Action | 8.5 | 1,530,000 | 2000 |
| 12 | Coco | Animation | 8.4 | 530,000 | 2017 |
| 13 | Forrest Gump | Drama | 8.8 | 2,100,000 | 1994 |
| 14 | Dune: Part Two | Sci-Fi | 8.6 | 420,000 | 2024 |
| 15 | La La Land | Romance | 8.0 | 600,000 | 2016 |

## # 4. RECOMMENDATION LOGIC & CRITERIA

Three robust recommendation strategies were implemented in Pandas:

1. **Criterion 1 (Global Highest Rating):** Evaluates universal quality by ordering records descending by `Rating`, utilizing `Votes` as secondary tie-breaker.
2. **Criterion 2 (User Preference / Genre Filtering):** Uses boolean masking (`df['Genre'] == user_genre`) to filter candidate items matching user preference, then ranks matches by rating.
3. **Criterion 3 (IMDb Weighted Rating Score):** Accounts for vote volume using the formula:

   `Weighted Score = [v / (v + m)] × R + [m / (v + m)] × C`

   where $v$ = votes, $m$ = 25th percentile threshold (695,000), $R$ = movie rating, and $C$ = dataset mean rating (8.61).

## # 5. PYTHON CODE IMPLEMENTATION

### Step 1: Import Libraries and Load Dataset

```
import os
import pandas as pd

# Load dataset from CSV into a Pandas DataFrame with fallback dictionary
def load_movie_dataset(csv_path="movies_dataset.csv"):
    if os.path.exists(csv_path):
        df = pd.read_csv(csv_path)
        print(f"[INFO] Loaded dataset successfully from '{csv_path}'")
    else:
        data = {
            "Movie_ID": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15],
            "Title": ["Inception", "The Dark Knight", "Interstellar", "Parasite", "Spirited Away",
                      "Avengers: Endgame", "The Shawshank Redemption", "Whiplash",
                      "Spider-Man: Into the Spider-Verse", "The Matrix", "Gladiator", "Coco",
                      "Forrest Gump", "Dune: Part Two", "La La Land"],
            "Genre": ["Sci-Fi", "Action", "Sci-Fi", "Drama", "Animation", "Action", "Drama", "Drama",
                      "Animation", "Sci-Fi", "Action", "Animation", "Drama", "Sci-Fi", "Romance"],
            "Rating": [8.8, 9.0, 8.7, 8.5, 8.6, 8.4, 9.3, 8.5, 8.4, 8.7, 8.5, 8.4, 8.8, 8.6, 8.0],
            "Votes": [2400000, 2700000, 1900000, 850000, 790000, 1180000, 2750000, 900000,
                      590000, 1950000, 1530000, 530000, 2100000, 420000, 600000],
            "Release_Year": [2010, 2008, 2014, 2019, 2001, 2019, 1994, 2014, 2018, 1999,
                             2000, 2017, 1994, 2024, 2016]
        }
        df = pd.DataFrame(data)
    return df
```

### Step 2: Criterion 1 — Highest Rating Recommendation Function

```
def recommend_top_by_rating(df, top_n=5):
    """Sorts DataFrame by Rating descending, Votes as tie-breaker."""
    recommended = df.sort_values(by=["Rating", "Votes"], ascending=[False, False]).head(top_n)
    display_df = recommended[["Title", "Genre", "Rating", "Votes", "Release_Year"]].reset_index(drop=Tru
e)
    display_df.index = display_df.index + 1
    return display_df
```

### Step 3: Criterion 2 — Genre Preference Recommendation Function

```
def recommend_by_genre(df, user_genre="Sci-Fi", top_n=3):
    """Filters by user preferred Genre and ranks by Rating."""
    filtered = df[df["Genre"].str.lower() == user_genre.strip().lower()]
    if filtered.empty:
        return f"No movies found for genre '{user_genre}'"
    recommended = filtered.sort_values(by=["Rating", "Votes"], ascending=[False, False]).head(top_n)
    display_df = recommended[["Title", "Genre", "Rating", "Votes", "Release_Year"]].reset_index(drop=Tru
e)
    display_df.index = display_df.index + 1
    return display_df
```

**Step 4: Criterion 3 — IMDb Weighted Rating Score Function**

```
def recommend_by_weighted_score(df, top_n=5):
    """Calculates Bayesian weighted score balancing rating & popularity."""
    C = df["Rating"].mean()
    m = df["Votes"].quantile(0.25)
    df_scored = df.copy()
    df_scored["Weighted_Score"] = (
        (df_scored["Votes"] / (df_scored["Votes"] + m)) * df_scored["Rating"] +
        (m / (df_scored["Votes"] + m)) * C
    ).round(2)
    recommended = df_scored.sort_values(by="Weighted_Score", ascending=False).head(top_n)
    display_df = recommended[["Title", "Genre", "Rating", "Votes", "Weighted_Score"]].reset_index(drop=T
rue)
    display_df.index = display_df.index + 1
    return display_df
```

**Step 5: Main Driver Program**

```
def main():
    print("=" * 65)
    print("    PANDAS MOVIE RECOMMENDATION SYSTEM - PRACTICAL 11")
    print("=" * 65)
    df = load_movie_dataset()
    print(f"Total Movies: {len(df)} | Columns: {list(df.columns)}")

    print("\n[CRITERION 1] TOP 5 HIGHEST RATED MOVIES:")
    print(recommend_top_by_rating(df, top_n=5).to_string())

    print("\n[CRITERION 2] TOP 3 RECOMMENDATIONS FOR 'SCI-FI':")
    print(recommend_by_genre(df, user_genre="Sci-Fi", top_n=3).to_string())

    print("\n[CRITERION 2] TOP 3 RECOMMENDATIONS FOR 'ANIMATION':")
    print(recommend_by_genre(df, user_genre="Animation", top_n=3).to_string())

    print("\n[CRITERION 3] TOP 5 RECOMMENDATIONS BY WEIGHTED SCORE:")
    print(recommend_by_weighted_score(df, top_n=5).to_string())
    print("=" * 65)

if __name__ == "__main__":
    main()
```

## # 6. PROGRAM EXECUTION OUTPUT (O/P)

**O/P**

```
=================================================================
    PANDAS MOVIE RECOMMENDATION SYSTEM - PRACTICAL 11
=================================================================
[INFO] Loaded dataset successfully from 'movies_dataset.csv'
Total Movies: 15 | Columns: ['Movie_ID', 'Title', 'Genre', 'Rating', 'Votes', 'Release_Year']

[CRITERION 1] TOP 5 HIGHEST RATED MOVIES:
                     Title   Genre  Rating    Votes  Release_Year
1  The Shawshank Redemption   Drama     9.3  2750000          1994
2           The Dark Knight  Action     9.0  2700000          2008
3                 Inception  Sci-Fi     8.8  2400000          2010
4              Forrest Gump   Drama     8.8  2100000          1994
5                The Matrix  Sci-Fi     8.7  1950000          1999
```

**1. Top 5 Movie Recommendations Output (Highest Rating)**

**O/P (Continued)**

```
[CRITERION 2] TOP 3 RECOMMENDATIONS FOR 'SCI-FI':
          Title   Genre  Rating    Votes  Release_Year
1     Inception  Sci-Fi     8.8  2400000          2010
2    The Matrix  Sci-Fi     8.7  1950000          1999
3  Interstellar  Sci-Fi     8.7  1900000          2014

[CRITERION 2] TOP 3 RECOMMENDATIONS FOR 'ANIMATION':
                            Title      Genre  Rating   Votes  Release_Year
1                     Spirited Away  Animation     8.6  790000          2001
2  Spider-Man: Into the Spider-Verse  Animation     8.4  590000          2018
3                              Coco  Animation     8.4  530000          2017
```

**2. Genre-Based Filtered Recommendations Output (Sci-Fi & Animation)**

**O/P**

```
[CRITERION 3] TOP 5 RECOMMENDATIONS BY WEIGHTED SCORE:
                      Title    Genre  Rating    Votes  Weighted_Score
1  The Shawshank Redemption    Drama     9.3  2750000            9.16
2           The Dark Knight   Action     9.0  2700000            8.92
3                 Inception   Sci-Fi     8.8  2400000            8.76
4              Forrest Gump    Drama     8.8  2100000            8.75
5              Interstellar   Sci-Fi     8.7  1900000            8.68
======================================================================
```

**3. Weighted Popularity & Rating Score Output**

## # 7. CONCLUSION & LEARNING OUTCOME

**Conclusion:** A comprehensive, lightweight recommendation system was successfully designed and developed. The system accurately processes multi-attribute data in Pandas DataFrames, performs composite sorting, handles ties gracefully, and offers user-centric filtering and mathematical scoring to produce Top 3 and Top 5 recommendations.

**Learning Outcomes Achieved:**

- Understood how structured tabular datasets are ingested, explored, and manipulated in Pandas.
- Implemented conditional subset filtering, multi-column sorting, and percentile calculations.
- Learned practical recommendation mechanisms used in commercial platforms (e.g., Netflix, IMDb).
- Processed structured data using Python and Pandas to generate recommendations for real-world applications.